МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №4

з курсу: «Мультиагентні системи»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Тема: Створення прикладової діалогової мультиагентної системи

Мета роботи: Ознайомитися з основними принципами та архітектурою прототипної мультиагенитної системи, яка наведена нижче як приклад, а також з програмними кодами на мові Python, і розробити проєкт власної діалогової МАС

Завдання:

Маючи досвід роботи з CrewAI, реалізувати власну діалогову систему, виконуючи кроки, наведені напочатку протоколу. Для цього необхідно використовувати інструменти (Tools), щоб логіку пошуку можна було загорнути у функцію-інструмент (декоратор @tool у CrewAI), яку агенти викликатимуть самостійно.

Необхідно створити мінімум два агенти: наприклад, "Менеджер з підтримки" (обробка запиту клієнта) та "Спеціаліст з бронювання" (працює з базою даних квитків).

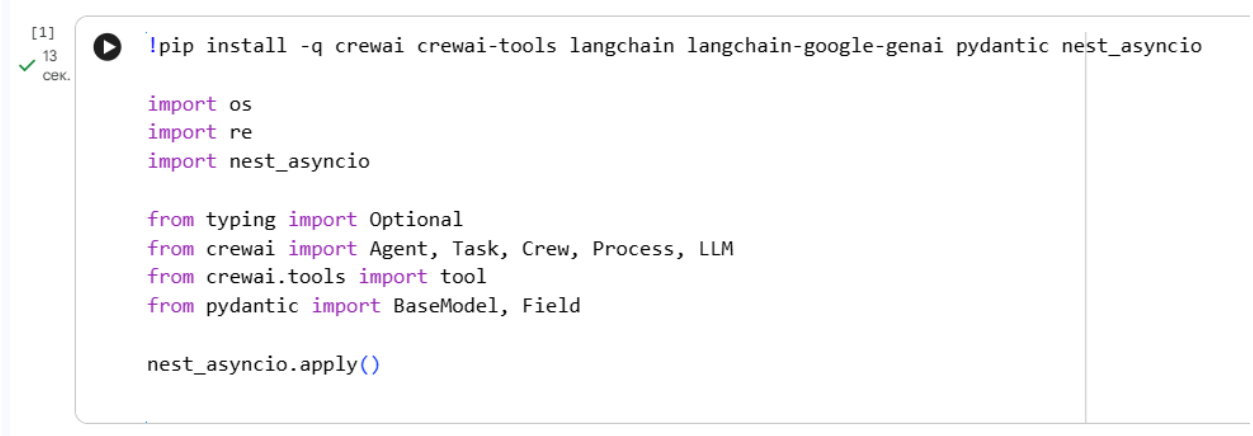
Варіант	**Предметна область**	**Сутності для витягання (Pydantic Schema)**	**Інструмент бази даних (@tool)**	**Приклад вхідного запиту користувача**
2	Бронювання столика в кафе	date, time, party_size, special_requests	check_table_availability(date, time, size) *Повертає номери вільних столиків та їх статус (біля вікна, тераса).*	*"Хочу замовити столик на 5 людей на п'ятницю на 19:00, бажано біля вікна."*

Хід роботи

1. Встановлення бібліотек

Спочатку було підготовлено середовище Google Colab. Для реалізації діалогової мультиагентної системи було використано бібліотеку CrewAI. Вона застосовується для створення агентів, задач, інструментів і запуску процесу взаємодії між агентами. У даній лабораторній роботі CrewAI використовується як основний фреймворк для побудови МАС.

Також було використано бібліотеку Pydantic, яка потрібна для опису структурованих схем даних. За її допомогою агент може повертати не просто звичайний текст, а об’єкт із чітко визначеними полями. Також підключено бібліотеку nest_asyncio. Вона потрібна для коректної роботи асинхронного запуску CrewAI у Google Colab. Оскільки Colab уже має активний цикл виконання, звичайний синхронний запуск міг викликати помилку, тому у фінальній версії використовується асинхронний запуск через kickoff async ().



2. Підключення API-ключа та мовної моделі

Далі було налаштовано доступ до мовної моделі Gemini. Для цього API ключ було записано у змінні середовища GOOGLE_API_KEY, GEMINI_API_KEY (щоб агенти зверталися до Gemini під час виконання задач)

Для роботи системи було створено об’єкт llm класу LLM (цей об’єкт передається агентам як мовна модель)

Параметр temperature було встановлено на значення 0.1. (щоб відповіді моделі були більш стабільними та менш випадковими), бо важливо, щоб агент витягував дату, час, кількість гостей і спеціальні побажання користувача.

```
os.environ.pop("GOOGLE_API_KEY", None)
os.environ.pop("GEMINI_API_KEY", None)

api_key = "AIzaSyBV3M-Rjlj6DZ3ii74gplsXpbVjmlBPJTU"

os.environ["GOOGLE_API_KEY"] = api_key
os.environ["GEMINI_API_KEY"] = api_key

llm = LLM(
    model="gemini/gemini-3-flash-preview",
    temperature=0.1
)
```

3. Створення Pydantic схем

Для роботи системи було створено дві Pydantic схеми: CafeReservationSchema та IncompleteCafeRequestSchema. Вони потрібні для того, щоб описати формат даних, які агент має отримати з природного запиту користувача

Схема CafeReservationSchema використовується для повного запиту, коли користувач вказав усі необхідні дані для бронювання столика. У цій схемі описано такі поля: date, time, party_size, special_requests та intent.

```python
class IncompleteCafeRequestSchema(BaseModel):
    date: Optional[str] = Field(
        default=None,
        description="Дата бронювання, наприклад: п'ятниця, завтра, 20.05.2026"
    )

    time: Optional[str] = Field(
        default=None,
        description="Час бронювання, наприклад: 19:00"
    )

    party_size: Optional[int] = Field(
        default=None,
        description="Кількість гостей"
    )

    special_requests: Optional[str] = Field(
        default=None,
        description="Особливі побажання клієнта, наприклад: біля вікна, тераса, VIP-зона"
    )

    intent: str = Field(
        ...,
        description="Намір користувача, наприклад: бронювання столика"
    )

    is_complete: bool = Field(
        ...,
        description="Чи достатньо даних для перевірки доступності столика"
    )

    clarification_question: str = Field(
        default="",
        description="Питання до користувача, якщо якихось даних бракує"
    )
```

Поле date зберігає дату бронювання, наприклад п’ятницю, time зберігає час бронювання, наприклад 19:00, party_size відповідає за кількість гостей. Поле special_requests містить особливі побажання користувача, напр біля вікна, на терасі або VIP зона, intent визначає намір користувача (показує, що він хоче саме забронювати столик).

Схема IncompleteCafeRequestSchema використовується тоді, коли користувач ввів неповний запит (якщо він написав тільки - Хочу забронювати столик на п’ятницю + не вказав час і кількість гостей. У цій схемі додатково є поля is_complete та clarification_question.

```python
class CafeReservationSchema(BaseModel):
    date: str = Field(
        ...,
        description="Дата бронювання, наприклад: п'ятниця, завтра, 20.05.2026"
    )

    time: str = Field(
        ...,
        description="Час бронювання, наприклад: 19:00"
    )

    party_size: int = Field(
        ...,
        description="Кількість гостей"
    )

    special_requests: Optional[str] = Field(
        default="немає",
        description="Особливі побажання клієнта, наприклад: біля вікна, тераса, VIP-зона"
    )

    intent: str = Field(
        ...,
        description="Намір користувача, наприклад: бронювання столика"
    )
```

Поле is_complete показує, чи достатньо даних для виконання бронювання. Якщо даних не вистачає, воно має значення False. Поле clarification_question містить уточнювальне питання, яке система повинна поставити користувачу (яку годину / яку кількість осіб потрібно забронювати столик)

4. Створення імітованої бази даних кафе

Потім було створено імітовану базу даних кафе. Вона представлена списком CAFE_TABLES, у якому зберігається інформація про доступні столики. Кожен столик описується окремим словником.

У кожного столика є такі характеристики: table_id, capacity, zone та status. Поле table_id містить номер столика, наприклад T1, T2, T3. Поле capacity показує, скільки гостей може вмістити цей столик. Поле zone описує місце розташування столика, наприклад зал, біля вікна, тераса або VIP зона. Поле status показує, що столик є вільним.

Також було створено словник BUSY_TABLES. Він потрібен для збереження інформації про зайняті столики на конкретну дату і час. Наприклад, у системі вказано, що на п’ятницю о 19:00 зайнятий столик T4, який знаходиться на терасі.

```
CAFE_TABLES = [
    {
        "table_id": "T1",
        "capacity": 2,
        "zone": "зал",
        "status": "вільний"
    },
    {
        "table_id": "T2",
        "capacity": 4,
        "zone": "зал",
        "status": "вільний"
    },
    {
        "table_id": "T3",
        "capacity": 6,
        "zone": "біля вікна",
        "status": "вільний"
    },
    {
        "table_id": "T4",
        "capacity": 6,
        "zone": "тераса",
        "status": "вільний"
    },
    {
        "table_id": "T5",
        "capacity": 8,
        "zone": "VIP-зона",
        "status": "вільний"
    }
]
```

```
BUSY_TABLES = {
    ("п'ятниця", "19:00"): ["T4"],
    ("завтра", "18:00"): ["T2"],
    ("20.05.2026", "20:00"): ["T3"]
}
```

По суті це спрощена БД, яка підходить для роботи, бо імітує зовнішню систему або API, до якого агент звертається для перевірки доступності столиків.

5. Функції нормалізації дати та часу

Також було створено допоміжні функції normalize_date та normalize_time. Вони потрібні для приведення введених користувачем даних до єдиного формату.

```
def normalize_date(date: str) -> str:
    date = date.lower().strip()
    date = date.replace("’", "'")

    if date.startswith("п'ятниц"):
        return "п'ятниця"

    return date


def normalize_time(time: str) -> str:
    time = time.strip()
    match = re.search(r"\d{1,2}:\d{2}", time)

    if match:
        return match.group(0)

    return time
```

Функція normalize_date обробляє дату, яку витягнув агент із запиту користувача. Наприклад, користувач може написати п’ятниця / п’ятницю. По суті це одне й те саме, але для програми це різні рядки. Тому функція перевіряє початок слова і приводить його до стандартного вигляду п'ятниця.

Функція normalize_time використовується для обробки часу. Вона шукає рядку час у форматі година-хвилини (19:00). Для цього застосовується регулярний вираз. Це дозволяє системі правильно працювати навіть тоді, коли користувач пише не просто 19:00, а, наприклад, о 19:00

6. Створення інструмента check_table_availability

Основним інструментом системи є функція check_table_availability. Вона оформлена за допомогою декоратора @tool, тому CrewAI сприймає її як інструмент, який агент може викликати самостійно під час виконання задачі.

```
@tool("check_table_availability")
def check_table_availability(date: str, time: str, size: int) -> str:
    """
    Перевіряє доступність столиків у кафе за датою, часом і кількістю гостей.
    Повертає номери вільних столиків та їхній статус: зал, біля вікна, тераса або VIP-зона.
    """

    normalized_date = normalize_date(date)
    normalized_time = normalize_time(time)

    busy_ids = BUSY_TABLES.get((normalized_date, normalized_time), [])

    available_tables = []

    for table in CAFE_TABLES:
        if table["capacity"] >= size and table["table_id"] not in busy_ids:
            available_tables.append(table)

    if not available_tables:
        return (
            f"На дату {date} о {time} немає вільних столиків "
            f"для {size} гостей. Можна запропонувати інший час."
        )

    result = f"Доступні столики на {date} о {time} для {size} гостей:\n"

    for table in available_tables:
        result += (
            f"Стіл {table['table_id']}: "
            f"місткість {table['capacity']} гостей, "
            f"зона: {table['zone']}, "
            f"статус: {table['status']}\n"
        )

    return result
```

Функція check_table_availability приймає три параметри: date, time та size. date - дату бронювання, time - час бронювання, size означає кількість гостей, для яких потрібно знайти столик.

Спочатку функція нормалізує дату і час за допомогою normalize_date та normalize_time. Потім вона перевіряє словник BUSY_TABLES, щоб визначити, які столики вже зайняті на вказану дату і час. Після цього функція проходить по списку CAFE_TABLES і відбирає тільки ті столики, які мають достатню місткість і не є зайнятими.

1. Якщо вільних столиків немає, функція повертає повідомлення про відсутність доступних варіантів.
2. Якщо вільні столики є, функція повертає їх список із номером столика, місткістю, зоною та статусом.

7. Створення агента менеджера з підтримки

Першим агентом у системі є nlu_agent. Він виконує роль менеджера з підтримки клієнтів. Його основна задача полягає в тому, щоб прочитати повідомлення користувача та визначити, які дані в ньому містяться.

```
nlu_agent = Agent(
    role="Менеджер з підтримки клієнтів",
    goal=(
        "Проаналізувати природний запит користувача щодо бронювання столика "
        "та визначити дату, час, кількість гостей і спеціальні побажання."
    ),
    backstory=(
        "Ти працюєш менеджером підтримки у кафе. "
        "Твоя задача полягає в тому, щоб уважно прочитати повідомлення клієнта, "
        "зрозуміти його намір і перетворити неструктурований текст "
        "у структуровані дані для подальшого бронювання."
    ),
    llm=llm,
    verbose=True
)
```

Агент відповідає за обробку природної мови. Він отримує вхідний текст користувача і витягує з нього дату бронювання, час, кількість гостей, спеціальні побажання та намір користувача. Наприклад - Хочу замовити столик на 5 людей на п’ятницю на 19:00, бажано біля вікна, агент повинен зрозуміти, що користувач хоче забронювати столик на п’ятницю о 19:00 для 5 гостей біля вікна. Наступні параметри:

1. role задається роль агента.
2. goal описується його основна мета.
3. backstory задається додатковий контекст (поведінка агента)

8. Створення агента спеціаліста з бронювання

Другим агентом у системі є booking_agent. Він виконує роль спеціаліста з бронювання столиків. Його задача полягає в тому, щоб отримати структуровані дані від першого агента, перевірити доступність столиків і сформувати відповідь для користувача.

До цього агента підключено інструмент check_table_availability, зроблено через параметр tools=[check_table_availability] (агент може самостійно викликати інструмент перевірки доступності столиків, коли йому потрібно отримати дані з бази)

```python
booking_agent = Agent(
    role="Спеціаліст з бронювання столиків",
    goal=(
        "Перевірити доступність столиків через інструмент бази даних "
        "і сформувати зрозумілу відповідь для клієнта."
    ),
    backstory=(
        "Ти відповідаєш за бронювання столиків у кафе. "
        "Ти маєш доступ до інструмента check_table_availability, "
        "який перевіряє наявність вільних столиків. "
        "Ти не повинен вигадувати результат самостійно, "
        "а маєш використовувати інструмент для перевірки доступності."
    ),
    tools=[check_table_availability],
    llm=llm,
    verbose=True
)
```

У параметрі backstory додатково вказано, що агент не повинен вигадувати результат самостійно. Він повинен використовувати інструмент бази даних (важливо продемонструвати саме роботу агента з інструментом)

9. Функція повного сценарію бронювання

Для повного сценарію було створено функцію run_complete _cafe_booking_ scenario. Вона використовується тоді, коли користувач надав усі необхідні дані для бронювання.

```python
async def run_complete_cafe_booking_scenario(user_input: str):
    task_extract = Task(
        description=(
            f'Проаналізуй вхідний текст користувача: "{user_input}". '
            "Виконай розпізнавання іменованих сутностей. "
            "Потрібно витягнути date, time, party_size, special_requests "
            "та визначити intent. "
            "Поверни повний структурований об'єкт CafeReservationSchema."
        ),
        expected_output=(
            "Структурований об'єкт CafeReservationSchema з полями "
            "date, time, party_size, special_requests, intent."
        ),
        agent=nlu_agent,
        output_pydantic=CafeReservationSchema
    )

    task_booking = Task(
        description=(
            "Використай дані, отримані на попередньому етапі. "
            "Передай у check_table_availability такі параметри: "
            "date, time та party_size як size. "
            "Після отримання результату інструмента врахуй special_requests користувача. "
            "Якщо користувач просить столик біля вікна, на терасі або у VIP-зоні, "
            "спочатку запропонуй столик з відповідною зоною. "
            "Якщо бажаний варіант недоступний, запропонуй найближчу альтернативу. "
            "Сформуй фінальну відповідь українською мовою."
        ),
        expected_output=(
            "Фінальна відповідь клієнту з результатом перевірки доступності столика, "
            "номером столика, місткістю, зоною та коротким поясненням."
        ),
        agent=booking_agent,
        context=[task_extract]
    )

    cafe_crew = Crew(
        agents=[nlu_agent, booking_agent],
        tasks=[task_extract, task_booking],
        process=Process.sequential,
        verbose=True
    )
```

Функція приймає один параметр user_input, тобто текстовий запит користувача. Усередині функції створюється перша задача task_extract. Вона передається агенту nlu_agent. У цій задачі агент повинен проаналізувати вхідний текст, витягнути сутності та повернути результат у форматі CafeReservationSchema.

Після цього створюється друга задача task_booking. Вона передається агенту booking_agent. Ця задача використовує результат першої задачі через context=[task_extract]. Це означає, що другий агент отримує дані, які були витягнуті першим агентом, і використовує їх для виклику інструмента check_table_availability.

Далі створюється об’єкт Crew. У нього передаються два агенти та дві задачі. Параметр process=Process.sequential означає, що задачі виконуються послідовно. Спочатку система аналізує запит користувача, а потім перевіряє доступність столиків.

10. Функція перевірки неповного запиту

Окремо було створено функцію run_incomplete_request_check. Вона використовується для перевірки ситуацій, коли користувач ввів не всі необхідні дані для бронювання.

```
async def run_incomplete_request_check(user_input: str):
    task_check = Task(
        description=(
            f'Проаналізуй вхідний текст користувача: "{user_input}". '
            "Визнач, чи хоче користувач забронювати столик у кафе. "
            "Витягни всі наявні дані: date, time, party_size, special_requests. "
            "Якщо якихось обов'язкових даних бракує, встанови is_complete=False "
            "і сформуй clarification_question українською мовою. "
            "Якщо всі дані є, встанови is_complete=True."
        ),
        expected_output=(
            "Структурований об'єкт IncompleteCafeRequestSchema з інформацією про те, "
            "чи достатньо даних для бронювання, і уточнювальним питанням за потреби."
        ),
        agent=nlu_agent,
        output_pydantic=IncompleteCafeRequestSchema
    )

    check_crew = Crew(
        agents=[nlu_agent],
        tasks=[task_check],
        process=Process.sequential,
        verbose=True
    )
```

Ця функція також приймає параметр user_input. Усередині неї створюється задача task_check, яку виконує агент nlu_agent. Агент повинен

визначити, які дані є у запиті, а яких не вистачає. Результат повертається у форматі IncompleteCafeRequestSchema.

Якщо користувач не вказав час або кількість гостей, агент встановлює is_complete=False. Після цього він формує уточнювальне питання в полі clarification_question.

11. Запуск основного сценарію

У першому запиті вкажемо усі необхідні дані (кількість гостей, день, час і спеціальне побажання), система може одразу виконати повний цикл роботи: проаналізувати запит, витягнути сутності, передати їх другому агенту, перевірити доступність столиків через інструмент і сформувати відповідь.

```
user_input_1 = "Хочу замовити столик на 5 людей на п'ятницю на 19:00, бажано біля вікна."

result_1 = await run_complete_cafe_booking_scenario(user_input_1)
```

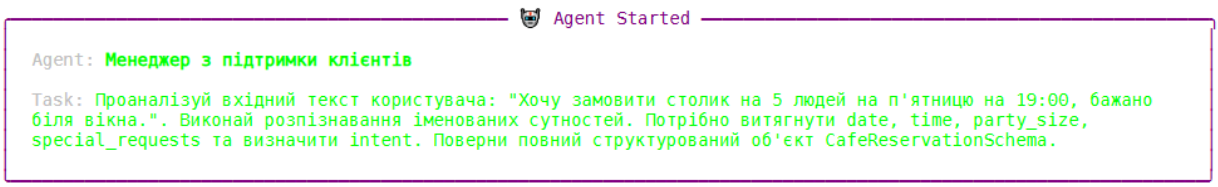
Результат :

Запуск системи та початок першої задачі. Починається виконання першої задачі task_extract (проаналізувати природний текст користувача і витягнути з нього потрібні сутності), тобто система повинна визначити дату бронювання, час, кількість гостей, спеціальні побажання та намір користувача.

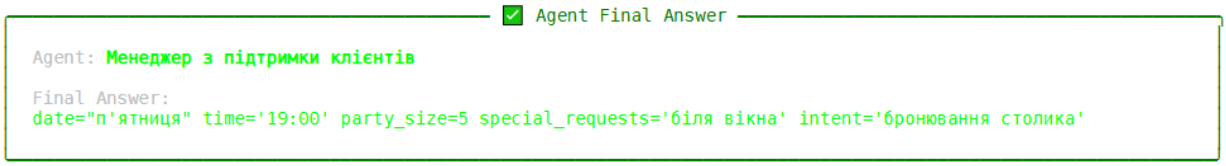
```
🚀 Crew Execution Started
Crew Execution Started
Name: crew
ID: 01ede215-af0a-42eb-b266-dc58a4ee96de

📋 Task Started
Task Started
Name: Проаналізуй вхідний текст користувача: "Хочу замовити столик на 5 людей на п'ятницю на 19:00, бажано
біля вікна.". Виконай розпізнавання іменованих сутностей. Потрібно витягнути date, time, party_size,
special_requests та визначити intent. Поверни повний структурований об'єкт CafeReservationSchema.
ID: 2eacf567-a39b-485e-b277-743ac4fa24d4
```

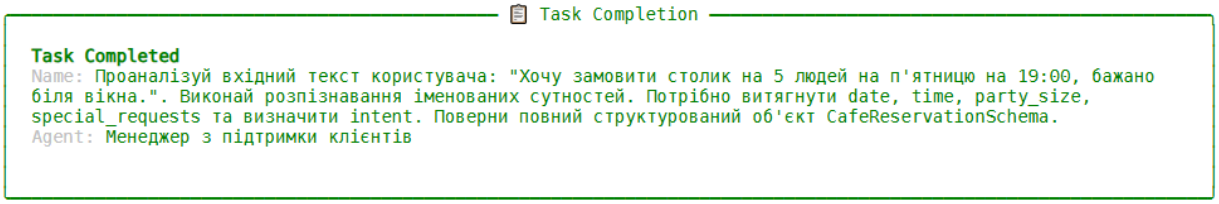
Після запуску задачі починає роботу перший агент (менеджер з підтримки клієнтів). Його роль полягає в обробці природної мови користувача. (читає повідомлення так, як його написала людина, і визначає, які дані в ньому є). У цьому сценарії агент має зрозуміти, що користувач хоче не просто поставити питання, а саме забронювати столик. Також він має знайти дату, час, кількість людей і побажання щодо місця біля вікна.



Видно фінальну відповідь першого агента. Він визначив, що дата бронювання це п'ятниця, час бронювання це 19:00, кількість гостей дорівнює 5, спеціальне побажання це біля вікна, а намір користувача це бронювання столика.

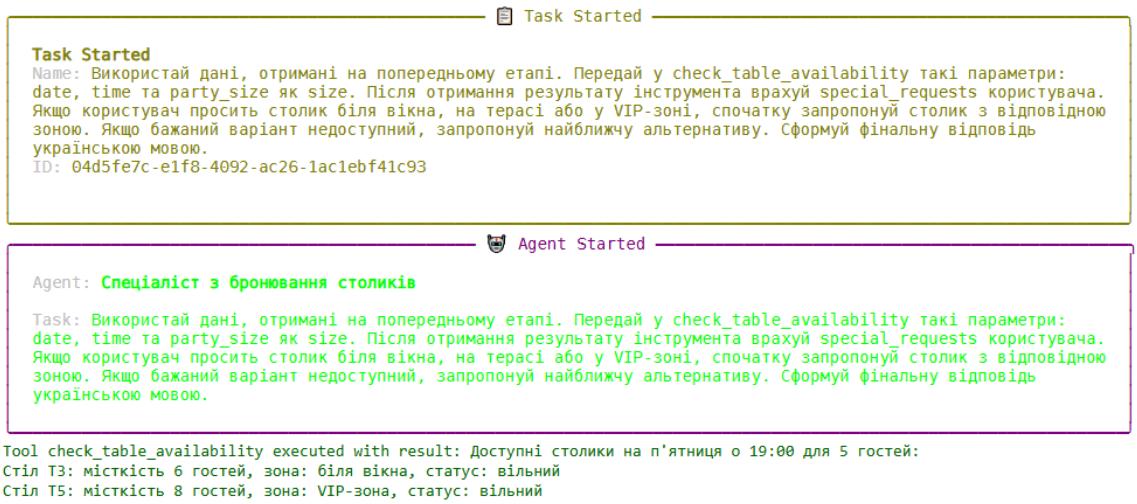


Менеджер з підтримки клієнтів виконав свою частину роботи і підготував структуровані дані для наступного етапу. Дані не втрачаються після завершення задачі, а передаються в наступну задачу через контекст context=[task_extract]. Завдяки цьому другий агент не починає роботу з нуля, а отримує вже підготовлені параметри бронювання.



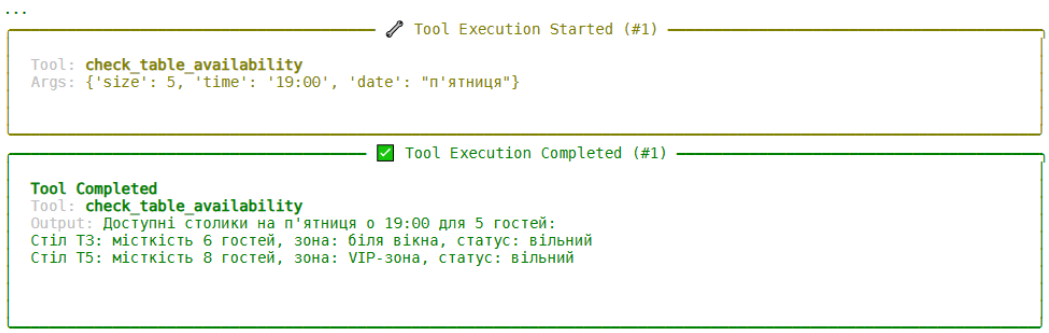
Після завершення першої задачі система переходить до другого етапу. У блоці Task Started запускається задача для агента Спеціаліст з бронювання столиків. Йому передаються дані, які раніше витягнув агент підтримки: дата, час, кількість гостей і побажання користувача. У цьому сценарії користувач хоче столик біля вікна, тому агент має врахувати це побажання під час вибору варіанта.

У блоці Agent Started видно, що до роботи приступив агент бронювання. Його задача полягає в тому, щоб не аналізувати текст заново, а використати вже готові структуровані дані та передати їх в інструмент check_table_availability.

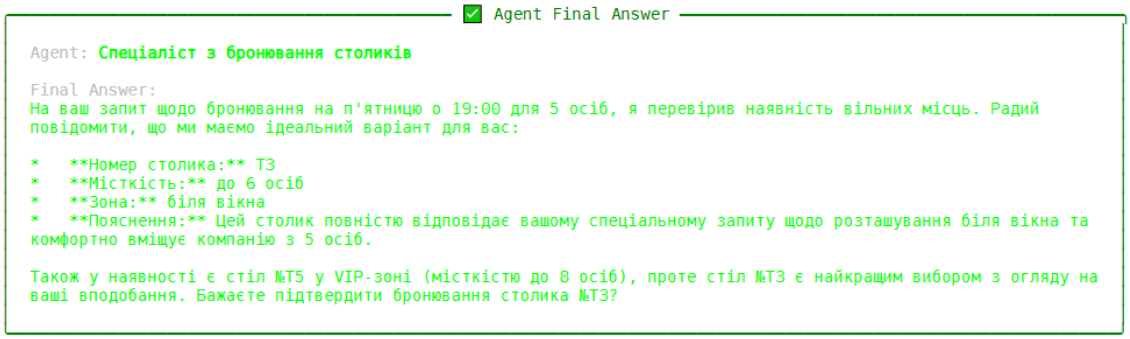


Далі агент Спеціаліст з бронювання столиків викликає інструмент check_table_availability. У Tool Execution Started видно, що в інструмент були передані параметри: кількість гостей 5, час 19:00 і дата п'ятниця.

У блоці Tool Execution Completed показано результат роботи інструмента. Функція перевірила імітовану базу даних кафе та знайшла два доступні варіанти: Т3 та Т5, (користувач просив місце біля вікна, найбільш відповідним варіантом для подальшої відповіді є саме столик Т3)



Система запропонувала столик Т3, оскільки він розрахований на 6 осіб, підходить для компанії з 5 гостей і знаходиться біля вікна. Також агент зазначив, що є альтернативний варіант Т5 у VIP зоні, але основним рекомендованим варіантом є саме Т3, бо він повністю відповідає запиту користувача.



12. Запуск сценарію коли бажаний столик зайнятий

У цьому сценарії користувач хоче забронювати столик на терасі для 6 гостей на п’ятницю о 19:00 (столик га терасі в цей час зайнятий, тому система має не підтвердити терасу, а запропонувати інші доступні варіанти)

```
user_input_2 = "Хочу забронювати місце на терасі для 6 гостей на п'ятницю о 19:00."

result_2 = await run_complete_cafe_booking_scenario(user_input_2)
```

Першою запускається задача аналізу вхідного тексту. Агент Менеджер з підтримки клієнтів читає повідомлення користувача та визначає основні дані для бронювання.

```
🚀 Crew Execution Started

Crew Execution Started
Name: crew
ID: cf2297cc-357e-4707-a368-5151f61d8d8e

📋 Task Started

Task Started
Name: Проаналізуй вхідний текст користувача: "Хочу забронювати місце на терасі для 6 гостей на п'ятницю о
19:00.". Виконай розпізнавання іменованих сутностей. Потрібно витягнути date, time, party_size,
special_requests та визначити intent. Поверни повний структурований об'єкт CafeReservationSchema.
ID: 589deab7-b830-4255-acbb-f41df6ca9050
```

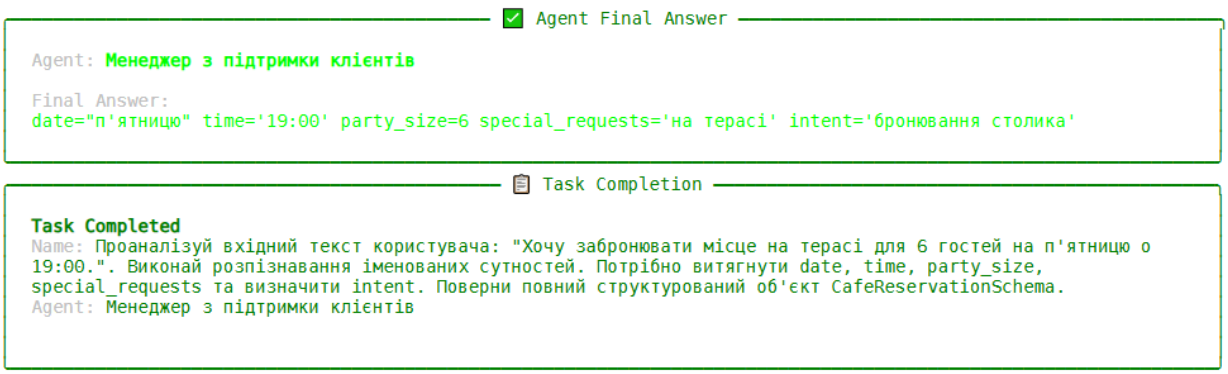
Далі йде запуска агента Менеджер з підтримки клієнтів. Його задача полягає в обробці природного тексту. Він повинен зрозуміти, що користувач хоче забронювати столик, і визначити дату, час, кількість гостей та спеціальне побажання.

```
🤖 Agent Started

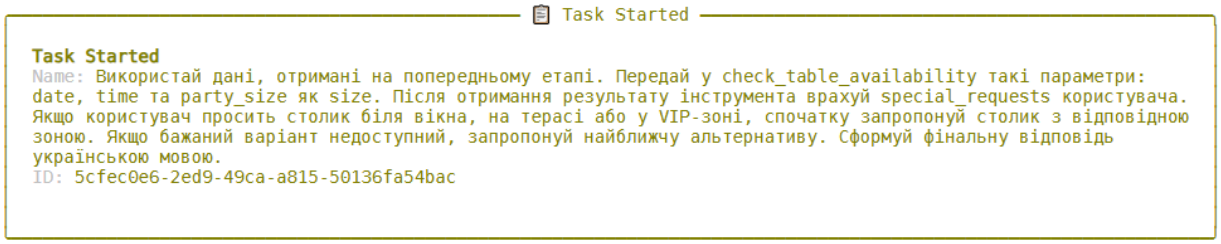
Agent: Менеджер з підтримки клієнтів

Task: Проаналізуй вхідний текст користувача: "Хочу забронювати місце на терасі для 6 гостей на п'ятницю о
19:00.". Виконай розпізнавання іменованих сутностей. Потрібно витягнути date, time, party_size,
special_requests та визначити intent. Поверни повний структурований об'єкт CafeReservationSchema.
```

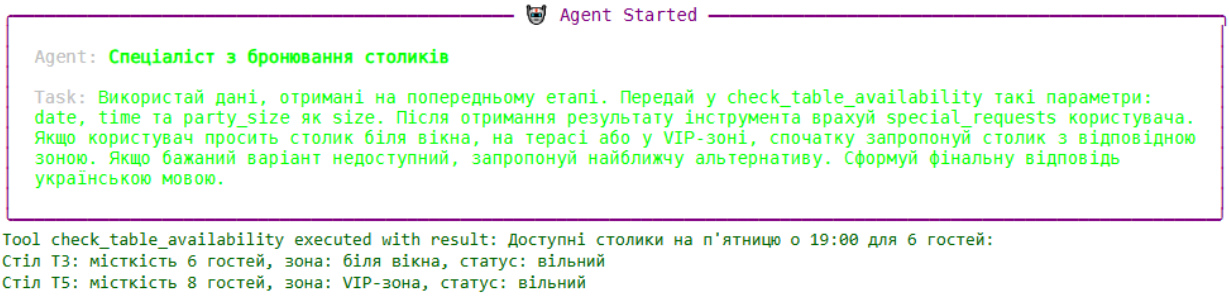
У наступному блоці можемо бачити, що перший агент правильно витягнув дані із запиту. Він визначив дату п'ятницю, час 19:00, кількість гостей 6, побажання на терасі та намір бронювання столика.



Після аналізу запиту запускається друга задача. Вона використовує готові дані, які були отримані від першого агента. Основна мета цього етапу: передати дату, час і кількість гостей в інструмент check_table_availability та врахувати побажання користувача щодо тераси.



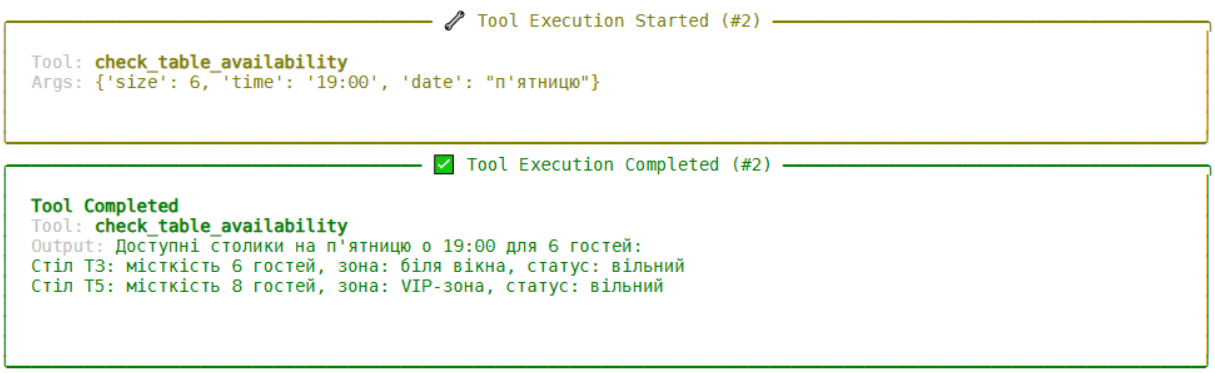
На наступному етапі починає працювати агент Спеціаліст з бронювання столиків. Він отримує дані від першого агента: п’ятниця, 19:00, 6 гостей і побажання “на терасі”. Його задача перевірити бд і знайти доступні столики.



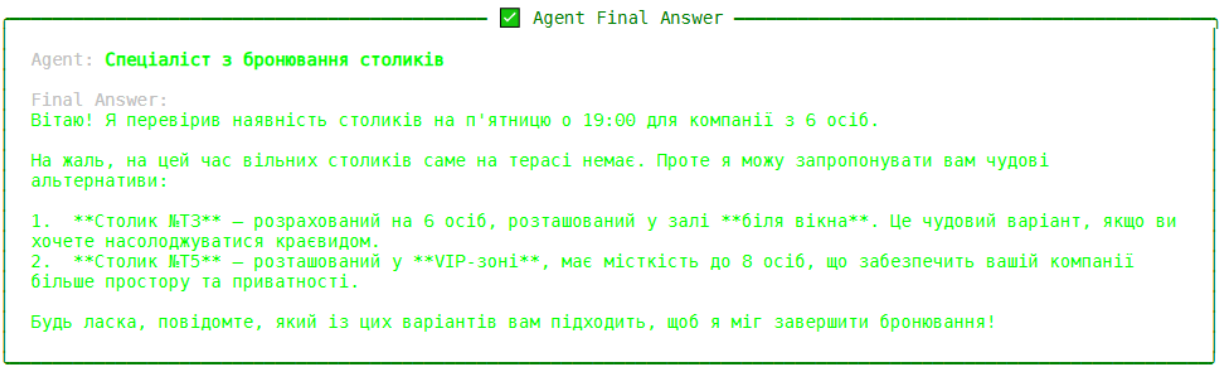

```
Tool check_table_availability executed with result: Доступні столики на п'ятницю о 19:00 для 6 гостей:
Стіл Т3: місткість 6 гостей, зона: біля вікна, статус: вільний
Стіл Т5: місткість 8 гостей, зона: VIP-зона, статус: вільний
```

Агент викликає інструмент check_table_availability. Він передає в нього параметри: кількість гостей 6, час 19:00 і дату п'ятницю (дані між агентами були передані правильно, перший агент витягнув параметри, а другий агент використав їх для перевірки доступності столиків)

Інструмент перевірив імітовану базу даних кафе та повернув два доступні столики: Т3 біля вікна та Т5 у VIP зоні.



Після отримання результату інструмента агент сформував відповідь користувачу. Він не підтвердив бронювання на терасі, бо такого вільного варіанта немає. Замість цього агент запропонував два доступні варіанти: Т3 біля вікна та Т5 у VIP зоні.



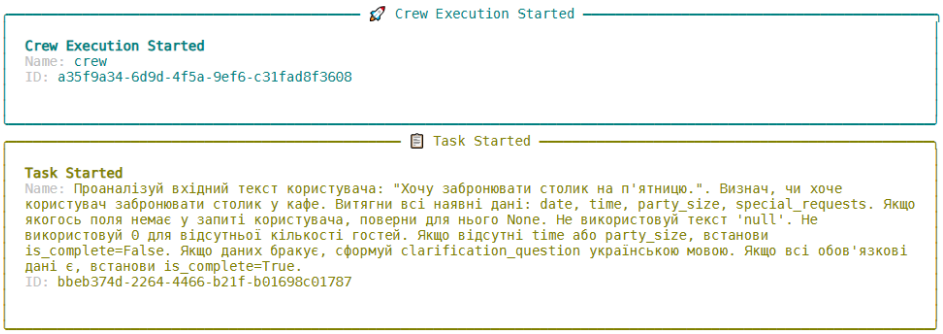
13. Запуск сценарію коли неповний запит

У цьому запиті користувач вказав тільки дату бронювання, але не вказав час і кількість гостей. Тому система не може одразу перевірити доступність столиків. Мета цього сценарію полягає в тому, щоб показати, що агент не вигадує відсутні дані, а формує уточнювальне питання.

```
user_input_3 = "Хочу забронювати столик на п'ятницю."

result_3 = await run_incomplete_request_check(user_input_3)
```

На першому етапі запускається окремий процес CrewAI для перевірки неповного запиту. У цьому сценарії використовується не повний процес бронювання з двома агентами, а тільки перевірка запиту через агента підтримки. Це зроблено тому, що без часу і кількості гостей неможливо коректно викликати інструмент перевірки столиків

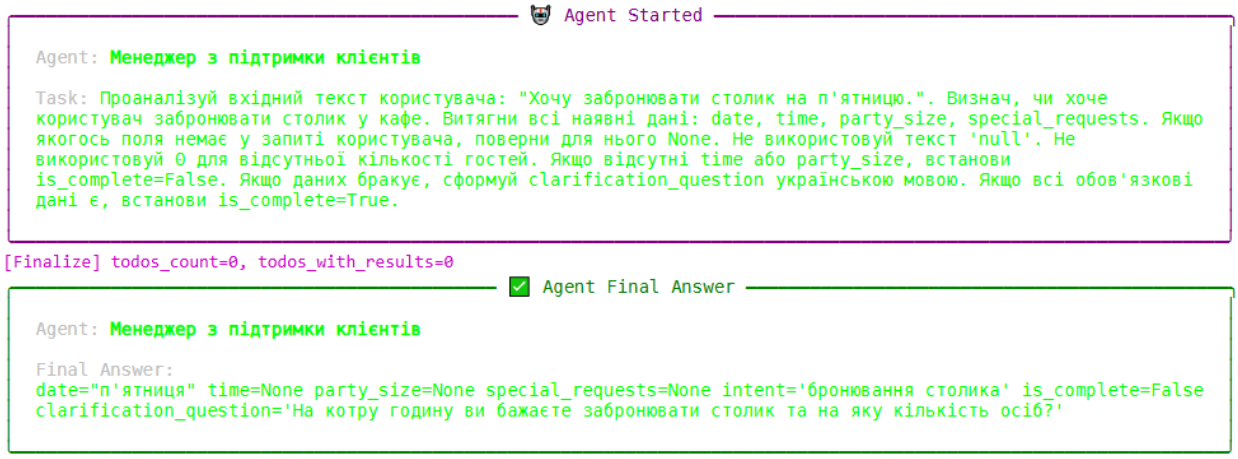


Далі відпрацьомує агент підтримки клієнтів, він читає повідомлення користувача, і перевіряє чи достатньо інформації для бронювання.

Агент правильно визначив наявні та відсутні дані. Він витягнув дату п'ятниця і визначив намір користувача як бронювання столика.

Водночас агент зрозумів, що у запиті немає часу бронювання, кількості гостей і спеціальних побажань. Тому поля time, party_size та special_requests отримали значення None.

Також агент встановив is_complete=False, тобто позначив запит як неповний. Після цього він сформував уточнювальне питання: годину і кількість осіб потрібно забронювати столик.



Тобто, у цьому сценарії не запускається агент бронювання і не викликається інструмент check_table_availability (без часу і кількості гостей система не може правильно перевірити доступність столиків)

Аналіз роботи системи

У процесі виконання лабораторної роботи було реалізовано прикладову діалогову мультиагентну систему для бронювання столика в кафе. Система працює з повідомленнями користувача природною мовою і перетворює їх на структуровані дані, які можна використати для перевірки доступності столиків.

Перший агент виконує роль менеджера з підтримки клієнтів. Він аналізує запит користувача, витягує потрібні сутності та визначає основний намір користувача. У нашій роботі агент виділяє дату бронювання, час, кількість гостей, спеціальні побажання та намір бронювання столика. Якщо запит користувача неповний, агент не вигадує відсутні значення, а формує уточнювальне питання.

Другий агент виконує роль спеціаліста з бронювання столиків. Він використовує результат першого агента, отримує структуровані дані та викликає інструмент check_table_availability. Цей інструмент перевіряє імітовану базу даних кафе та повертає доступні столики відповідно до дати, часу і кількості гостей.

Використання Pydantic схем дозволило отримувати структурований результат аналізу запиту. Завдяки цьому дані між агентами передаються у зручному форматі, а система може чітко розуміти, які параметри вже є, а яких ще не вистачає. Для повного запиту використовується схема CafeReservationSchema, а для неповного запиту використовується IncompleteCafeRequestSchema.

Використання інструмента check_table_availability показує, як агент може працювати із зовнішнім джерелом даних. У цій лабораторній роботі база даних була змодельована у вигляді списку столиків та словника зайнятих бронювань. У реальній системі замість тестової бази цей інструмент міг би звертатися до бази даних кафе або до зовнішнього API бронювання.

Система показала здатність працювати з різними типами запитів. У першому сценарії користувач одразу вказав усі необхідні дані, тому система знайшла відповідний столик біля вікна. У другому сценарії бажаний столик на терасі був зайнятий, тому система запропонувала альтернативні варіанти. У третьому сценарії запит був неповним, тому система не запускала перевірку столиків, а поставила уточнювальне питання.

Висновок

У ході виконання лабораторної роботи було реалізовано прикладову діалогову мультиагентну систему для бронювання столика в кафе. Система приймає запит користувача природною мовою, аналізує його та перетворює на структуровані дані: дату, час, кількість гостей, спеціальні побажання та намір користувача. Для цього було використано фреймворк CrewAI, мовну модель Gemini та Pydantic схеми, які дозволяють отримувати результат у чіткому форматі.

У системі було створено двох агентів. Перший агент виконує роль менеджера з підтримки клієнтів і відповідає за аналіз запиту та витягнення потрібних сутностей. Другий агент виконує роль спеціаліста з бронювання столиків, використовує результат першого агента та викликає інструмент check_table_availability. Цей інструмент імітує роботу бази даних кафе та перевіряє, які столики доступні за заданими параметрами.

Результати запусків показали, що система коректно працює з різними типами запитів. У повному запиті вона змогла підібрати столик біля вікна, у сценарії із зайнятою терасою запропонувала альтернативні варіанти, а у випадку неповного запиту сформувала уточнювальне питання.

Додаток А

Посилання на Google Colab:
https://colab.research.google.com/drive/1jFEODFAfoyWdwe5KhvOB_UUrrAYpLsDU?usp=sharing